

PHASE 8 — Reports, Analytics, Settings & Final Polish

PRSECURITY HRMS

IMPORTANT RULES

Output every file 100% completely. Never use '...' or 'rest remains same'.

After each file write "--- FILE COMPLETE ---" then start the next file immediately.

If about to hit output limit, finish current file and write "--- PAUSED: Type CONTINUE ---"

This is Phase 8 of 8 — the FINAL phase. All previous modules are complete.

DO NOT rebuild previous files. Only add NEW files or EXTEND where specified.

CONTEXT

Project: PRSECURITY HRMS | Stack: React+Vite+TS+Tailwind+shadcn/ui |

Node+Express+Prisma+PostgreSQL Roles: ADMIN, HR, EMPLOYEE, INTERN | Currency: ₹ This phase completes the entire HRMS with reports, admin settings, profile page, dark mode, and final polish.

PART A — REPORTS & ANALYTICS

STEP A1 — Reports Backend

FILE: backend/src/services/reports.service.ts

getAttendanceReport(month, year, userId?):

Monthly attendance summary per employee (or single if userId provided)

Returns: array of { employee: {id, name, employeeId}, workingDays, presentDays, absentDays, halfDays, leaveDays, attendancePercent }

Also: daily trend data (array of { date, presentCount, absentCount } for the month) for chart

getPayrollReport(month, year):

Returns: array of { employee, grossSalary, netSalary, pfEmployee, esiEmployee, tds, bonus, paymentStatus }

Summary totals: totalGross, totalNet, totalPF, totalESI, totalTDS, totalBonus

Monthly trend: array of { month, year, totalNet } for last 12 months

getLeaveReport(year):

Leave utilization by type: array of { leaveType, totalAllocated, totalUsed, utilizationPercent }

Per employee: array of { employee, casualUsed, sickUsed, earnedUsed, ... }

Monthly leave trend: array of { month, totalLeaveDays }

getHeadcountReport():

Current: totalActive, totalInactive, totalTerminated

By role: array of { role, count }

By employment type: array of { type, count }

Monthly joining trend (last 12 months): array of { month, year, joined, left }

getTaskReport(userId?):

Task completion by employee: array of { employee, total, done, inProgress, overdue, completionRate }

Overall: counts by status, avg completion time

`getProjectReport()`:

Projects by status: array of { status, count }

Projects list with: name, client, status, tasksTotal, tasksDone, progressPercent, daysRemaining

`exportToCSV(data, filename)`:

Generic CSV export utility function

Converts array of objects to CSV string

Returns CSV content

FILE: backend/src/controllers/reports.controller.ts

FILE: backend/src/routes/reports.routes.ts

```
□  
  
GET /api/reports/attendance → getAttendanceReport?month=&year=&userId= (HR/Admin)  
GET /api/reports/payroll → getPayrollReport?month=&year= (HR/Admin)  
GET /api/reports/leave → getLeaveReport?year= (HR/Admin)  
GET /api/reports/headcount → getHeadcountReport (HR/Admin)  
GET /api/reports/tasks → getTaskReport?userId= (HR/Admin)  
GET /api/reports/projects → getProjectReport (HR/Admin)
```

STEP A2 — Reports Frontend

FILE: frontend/src/api/reports.api.ts

FILE: frontend/src/utils/exportCSV.ts

```
□  
  
typescript  
export function exportToCSV(data: Record<string, any>[], filename: string): void  
// Converts data array to CSV, triggers browser download
```

FILE: frontend/src/pages/reports/ReportsPage.tsx

HR/Admin only. Full reports dashboard with 6 sections using tabs:

Tab 1 — Headcount:

StatCards: Total Active, Total This Month (joined), Total Left This Month

Pie chart (Recharts): by Role

Pie chart: by Employment Type

Line chart: Monthly joining trend (last 12 months)

Export CSV button

Tab 2 — Attendance:

Month/Year selector

Employee filter (optional)

Bar chart: Present vs Absent vs Leave per day of month

DataTable: Employee, Working Days, Present, Absent, Half Day, Leave, Attendance %

Export CSV

Tab 3 — Leave:

Year selector

Bar chart (grouped): Leave type utilization (allocated vs used) per type

DataTable: Employee + columns per leave type showing used/remaining

Export CSV

Tab 4 — Payroll:

Month/Year selector

Summary cards: Total Gross (₹), Total Net (₹), Total PF (₹), Total ESI (₹), Total TDS (₹)

Bar chart: Net salary by employee

Line chart: Monthly total payroll trend (last 12 months)

DataTable: Employee, Gross, Deductions, Net, Status

Export CSV

Tab 5 — Tasks:

Bar chart (stacked): Tasks by status per employee

DataTable: Employee, Total Tasks, Done, In Progress, Overdue, Completion Rate %

Export CSV

Tab 6 — Projects:

Pie chart: Projects by status

DataTable: Project name, Client, Status, Tasks Done/Total, Progress %, Days Remaining

Export CSV

All charts use Recharts with the primary color palette (`#1E3A5F`, `#00C2FF`). All DataTables use the shared DataTable component.

PART B — SETTINGS MODULE (Admin Only)

STEP B1 — Settings Backend

FILE: backend/src/services/settings.service.ts

`getCompanySettings()`:

Return company settings from a Settings table (or use a simple key-value store in DB)

Returns: companyName, address, gstin, financialYearStart (month number), logoUrl, phone, email

`updateCompanySettings(data, adminId)`:

Update settings

`getHolidays(year)`:

Return all holidays for year

`createHoliday(data)`:

Add holiday: name, date, type (NATIONAL/OPTIONAL/COMPANY)

`updateHoliday(id, data)`:

Update

`deleteHoliday(id)`:

Delete

`getAllUsers(query)`:

Admin: all users with role, status, last login info

Paginated, filter by role/status

`updateUserRole(userId, role, adminId)`:

Change user role (Admin only, cannot change own role)

`deactivateUser(userId, adminId)`:

Set status=INACTIVE

`resetUserPassword(userId, adminId)`:

Reset to temp password: Prs@{employeeId}

Return temp password

`getAuditLogs(query)`:

Paginated list of system actions

Filter by action type, userId, dateRange

Note: Create a Settings model and AuditLog model in Prisma:

```
prisma

model Settings {
  id          String @id @default("1")
  companyName String @default("PRSECURITY CONSULTANCY & SERVICES")
  address     String?
  gstin       String?
  phone       String?
  email       String?
  logoUrl     String?
  financialYearStart Int @default(4)
  updatedAt   DateTime @updatedAt
}

model AuditLog {
  id          String @id @default(uuid())
  userId      String
  action      String
  entity      String
  entityId    String?
  details     String?
  ipAddress   String?
  user        User   @relation(fields: [userId], references: [id])
  createdAt   DateTime @default(now())
}
```

Run: `npx prisma migrate dev --name add_settings_audit`

Add audit logging middleware that auto-logs HR/Admin POST/PUT/DELETE actions.

FILE: backend/src/controllers/settings.controller.ts

FILE: backend/src/routes/settings.routes.ts

GET /api/settings/company → getCompanySettings (Admin)
PUT /api/settings/company → updateCompanySettings (Admin)
GET /api/settings/holidays → getHolidays?year= (authenticated)
POST /api/settings/holidays → createHoliday (Admin)
PUT /api/settings/holidays/:id → updateHoliday (Admin)
DELETE /api/settings/holidays/:id → deleteHoliday (Admin)
GET /api/settings/users → getAllUsers (Admin)
PUT /api/settings/users/:id/role → updateUserRole (Admin)
PUT /api/settings/users/:id/deactivate → deactivateUser (Admin)
POST /api/settings/users/:id/reset-password → resetUserPassword (Admin)
GET /api/settings/audit-logs → getAuditLogs (Admin)

STEP B2 — Settings Frontend

FILE: frontend/src/api/settings.api.ts

FILE: frontend/src/pages/settings/SettingsPage.tsx

Admin only. Tabbed settings page:

Tab 1 — Company:

Form: Company Name, Address, GSTIN, Phone, Email, Financial Year Start (month dropdown), Logo Upload

Save button

Tab 2 — Holidays:

Year selector

Table: Holiday Name, Date, Type badge, Actions (Edit, Delete)

"Add Holiday" button → modal: Name, Date, Type

Seeded holidays shown with edit/delete

Tab 3 — Payroll Settings:

Form: Default PF Employee %, Default PF Employer %, Default ESI Employee %, Default ESI Employer %, Default TDS %, Payroll Processing Day (1-28)

Default TDS %, Payroll Processing Day (1-28)

These become defaults when creating new employee salary structures

Tab 4 — Leave Policy:

Table of all leave types with edit inline

Columns: Name, Code, Default Days, Paid, Carry Forward, Max Carry Forward, Actions

"Add Leave Type" button

Tab 5 — User Management:

DataTable: Employee ID, Name, Email, Role badge, Status badge, Last Active, Actions

Actions: Change Role (dropdown), Reset Password (shows temp password in modal), Deactivate

Filters: Role, Status

Tab 6 — Audit Log:

Read-only DataTable: Timestamp, User, Action, Entity, Entity ID, IP Address

Filter by date range, user

Export CSV

PART C — PROFILE PAGE

FILE: frontend/src/pages/profile/ProfilePage.tsx

All roles. Self-service profile management:

Section 1 — Profile Picture:

Current picture or initials avatar

Upload new picture button → cropper modal (use react-image-crop)

Remove picture option

Section 2 — Personal Information:

Display: Name, Email, Employee ID, Designation, Role, Status, Joining Date

Editable (click Edit): Phone, Date of Birth, Address, Emergency Contact Name, Emergency Contact Phone

Save button

Section 3 — Bank Details:

Display masked: Account Number (last 4 digits), IFSC, Bank Name, PAN (masked)

Edit button → form to update

Section 4 — Change Password:

Current Password, New Password, Confirm New Password

Zod validation: min 8 chars, 1 uppercase, 1 number, 1 special char

Save button

Section 5 — My Assets (if any assigned):

Simple list of assets assigned to the user

PART D — DARK MODE

Update frontend/src/store/themeStore.ts (new file)

```

typescript
interface ThemeStore {
  isDark: boolean
  toggleDark: () => void
}
// Persists to localStorage
// On init: apply 'dark' class to document.documentElement if isDark
```

Update frontend/src/components/layout/Topbar.tsx

Connect dark mode toggle to themeStore

On toggle: add/remove 'dark' class on document.documentElement

Ensure all pages and components use Tailwind dark: variants consistently:

Background: bg-white dark:bg-slate-800

Cards: bg-white dark:bg-slate-900

Text: text-slate-900 dark:text-slate-100

Borders: border-slate-200 dark:border-slate-700

Sidebar: bg-[#1E3A5F] (stays dark in both modes — it's already dark)

PART E — FINAL POLISH & MISSING CONNECTIONS

Update frontend/src/pages/employees/EmployeeDetailPage.tsx

Connect all remaining tabs with real data:

Leave tab: show employee's leave balance + request history

Payslips tab: show employee's payslip list with download buttons

Tasks tab: show tasks assigned to employee (mini list)

Update frontend/src/pages/dashboard/AdminDashboardPage.tsx

Connect all remaining StatCards:

pendingLeaveRequests → real count

Add chart: Monthly Payroll Trend (last 6 months) using payroll report data

Update frontend/src/App.tsx

Register the new routes:

```
☐
/reports    → ReportsPage (HR/Admin)
/assets     → AssetsPage (HR/Admin)
/expenses   → ExpensesPage (authenticated)
/announcements → AnnouncementsPage (authenticated)
/helpdesk   → HelpdeskPage (authenticated)
/settings   → SettingsPage (Admin)
/profile    → ProfilePage (authenticated)
```

(Replace the placeholder components from Phase 1 with the real pages)

FILE: backend/src/app.ts (update — add missing routers)

Mount all new routers added in phases 5–8:

```
☐
typescript
app.use('/api/clients', clientRoutes)
app.use('/api/projects', projectRoutes)
app.use('/api/tasks', taskRoutes)
app.use('/api/jobs', jobRoutes)
app.use('/api/applicants', applicantRoutes)
app.use('/api/interviews', interviewRoutes)
app.use('/api/performance', performanceRoutes)
app.use('/api/assets', assetRoutes)
app.use('/api/expenses', expenseRoutes)
app.use('/api/announcements', announcementRoutes)
app.use('/api/tickets', helpdeskRoutes)
app.use('/api/reports', reportsRoutes)
app.use('/api/settings', settingsRoutes)
```

PART F — FINAL README UPDATE

Update README.md (full version)

Structure:

Project Overview — what PRSECURITY HRMS is, tech stack summary

Prerequisites — Node 18+, PostgreSQL 15, pnpm or npm

Quick Start with Docker

```
bash
docker-compose up -d
```

Backend Setup

```
bash
cd backend
npm install
cp .env.example .env
# Edit .env with your values
npx prisma migrate dev
npx prisma db seed
npm run dev
```

Frontend Setup

```
bash
cd frontend
npm install
cp .env.example .env
npm run dev
```

Default Login Credentials

Admin: [admin@prsecurity.in](#) / Admin@1234

HR: [hr@prsecurity.in](#) / Hr@1234

Module Overview — one-line description per module

API Base URL — <http://localhost:5000/api>

Key Environment Variables — table of all vars with description

Folder Structure — full tree

Role Permissions Matrix — table: Module vs Role showing access level

Payroll Calculation Formula — documented

Default Leave Types — table of all 8 types

Troubleshooting — common issues (DB connection, port conflicts, Prisma migration errors)

AFTER ALL FILES ARE COMPLETE

Write this exactly: " 🇮🇳 PRSECURITY HRMS — ALL 8 PHASES COMPLETE!

Your full HRMS is ready with: ☒ Authentication & Role-based Access (4 roles) ☒ Employee Management (profiles, documents, salary) ☒ Attendance (punch in/out with geolocation, monthly logs) ☒ Leave Management (8 types, apply/approve, team calendar) ☒ Payroll (PF/ESI/TDS auto-calculation, PDF payslips in ₹) ☒ Task Management (Kanban board, drag-and-drop) ☒ Project & Client Management (teams, timeline, progress) ☒ Recruitment ATS (pipeline, interviews, hire-to-employee) ☒ Performance Management (goals, review cycles) ☒ Asset Management (assign/return tracking) ☒ Expense Claims (submit, approve, pay) ☒ Announcements (role-targeted, priority levels) ☒ Helpdesk (ticket raise,

assign, resolve) ☒ Reports & Analytics (6 report types with charts + CSV export) ☒ Admin Settings
(company, holidays, users, audit log) ☒ Dark Mode, Mobile Responsive, Real-time Notifications
Run docker-compose up -d, seed the DB, and start building! 🚀 "